

AI-augmented static analysis framework for secure C software

Amadéo Gilliéron^{1,2}[0009–0004–9896–9609], Nga Nguyen²[0000–0003–3273–8272],
Youssef Souissi¹[], and Guillaume Guerard²[0000–0002–6773–221X] Ville
Yli-Mäyry¹[0000–0002–2435–0350]

¹ Secure-IC S.A.S, 801 avenue des Champs Blancs, 35510 Cesson-Sévigné, France
`contact@secure-ic.com`
<https://www.secure-ic.fr/>

² De Vinci Research Center, De Vinci Higher Education, 12 Avenue Léonard de Vinci
92916 PARIS LA DEFENSE CEDEX, France
`contact@devinci.fr`

Abstract. MISRA-C is a set of rules and recommendations intended to regulate the use of the C language in order to improve software reliability and maintainability. It is particularly adopted in safety-critical embedded systems, where failures caused by ambiguous or undefined language behaviors may lead to severe consequences. In the state of the art, the detection of MISRA-C rule violations has been extensively studied, notably through mature static-analysis tools capable of efficiently identifying non-compliances. However, the automatic correction of these violations remains largely an open problem: existing approaches are partial and often limited to simple rules, or require significant human intervention. Consequently, this topic remains an active research area that is still being structured and deepened. In this context, this paper presents a framework for the automatic correction of MISRA-C violations. The framework aims to integrate generative artificial intelligence into the software compliance process. It leverages large language models (LLMs) to iteratively generate and refine fixes in the C language. This approach is combined with static-analysis and immediate feedback mechanisms, ensuring compliance with MISRA-C rules while preserving the functional behavior of the code. The correction framework is evaluated using ExeBench, a dataset containing executable C functions with input/output (IO) pairs. The IO specification is used to assess behavioral correctness, while compilation ensures syntactic validity. Using ExeBench as a baseline provides a standardised and executable evaluation framework, ensuring that the results are comparable across studies, and readily exploitable by future work. To assess its applicability beyond isolated functions, the framework is also evaluated on an industrial-scale project, where it addresses the practical challenges of large-scale software compliance.

Keywords: Software Security · Vulnerabilities · Automated Program Repair · Implementations · Formal Methods

1 Introduction

The C language is widely used in the development of embedded and critical systems. Its high performance and low level of abstraction with a small code footprint makes it well suited for developing for devices with hardware constraints. However, the low level of abstraction and the lack of built-in safeguards in the C language allow for many undefined, ambiguous, or implementation-dependent behaviors. These characteristics are a frequent source of subtle errors that are difficult to control, causing potential safety or security risks. Thus, the problem of accurate C code correction is an ongoing research challenge.

In this context, the MISRA-C [12] standard aims to supervise the use of the language by defining a set of rules and recommendations designed to limit these sources of error and improve the reliability and maintainability of the code. By explicitly targeting the most risky language constructs, MISRA-C provides a structured and formalised environment for analysing and correcting non-conformities, while remaining closely linked to the constraints and realities of software development in C. This standard is widely used and referenced in high-level frameworks such as ISO 26262 [1] (up to ASIL-D) for automotive systems, as well as by organisations and authorities such as ANSSI [2] in the context of secure software development in C. While violation detection can rely on formalised static analysis, the correction phase remains largely manual and time-consuming. Automated Program Repair (APR) is a major challenge due to the complexity and diversity of software projects. Automatic patch generation must simultaneously preserve existing functional behavior and respect the specific constraints of the language and application domain. It raises several intrinsic difficulties:

- Accurately identify the portions of code affected by a violation across a potentially large and modular project.
- Apply changes reliably across the entire source code, without unnecessary additions or modifications.
- Meet the specific requirements and standards of the field and project.

In the context of MISRA-C, additional constraints arise:

- Comply with the granularity and the strict restrictions of the standard that limit or prohibit certain C language idioms (expressions with multiple side effects, pointer arithmetic).
- Maintain overall project compliance, as a local correction may cause violations elsewhere in the code, sometimes requiring a broader scope of the project to be taken into account.

To overcome the above problems, we propose an AI-based correction tool that can be used to correct MISRA-C compliance violations. In the correction cycle, we use the static analyser’s report to associate rule violations to specific objects in the code. Then, the correction framework provides the LLM with curated examples and guidance on MISRA-C rules to meet developers’ expectations. The targeted MISRA-C rules are those whose incorrect correction could modify the

program’s behavior; rules that can be fixed through simple hard-coded transformations are not considered. While in this paper we consider MISRA-C standard, this framework can be applied to other coding ruleset such as CERT-C [14] for instance. We evaluate the tool performances on ExeBench executable C functions [3] as a baseline exploitable by future work. The framework was also tested on an internal industrial project to validate its functionality on large codebases.

This paper is organised as follows. Section 2 reviews the state of the art, including existing approaches for detecting MISRA-C rule violations and automated correction techniques, and discusses their limitations. Section 3 presents the proposed method, which includes a context module, a correction loop, and a test-in-the-loop mechanism for validating corrections. The results of the approach are given in Section 4. Finally, Section 5 concludes the paper and outlines directions for future work.

2 State of the art

2.1 Detection of MISRA-C rule violations

The detection of MISRA-C rule violations is now widely supported by a variety of tools. It is mainly based on static analysis techniques such as control flow analysis, data flow analysis, and symbolic execution. Mature industrial solutions such as Parasoft C/C++test, Coverity [16] and Polyspace offer extensive coverage of the MISRA-C standard. They are heavily used in critical industrial contexts, providing detailed diagnostics, traceability mechanisms, and compliance reports. In addition, open-source or academic tools, such as FRAMA-C [6], GCC-MISRA-C Checker [5], Cppcheck, or Clava-MISRA [10], offer alternative approaches based on existing compilers or code transformation infrastructures. These tools are quicker to understand and to modify, making them easier to integrate and use in research.

2.2 Automated correction

The field of automatic code correction aims to generate software fixes from defective programs with limited human intervention. Recent developments in the field have been largely driven by the adoption of LLMs, which have given rise to a variety of correction approaches based on generative AI [4]. Procedural approaches are based on a fixed correction pipeline in which the overall sequence of steps is explicitly defined by the framework. In this context, LLMs are integrated as components of the pipeline through mechanisms that include human feedback or test feedback and prompting techniques. These approaches allow for iterative refinement while maintaining strong external control. Less guided approaches, known as agentic approaches, dynamically delegate the choice and orchestration of the various correction steps to the LLM [17]. Agentic frameworks give one or more LLMs the ability to plan, generate, validate, and revise corrections

autonomously, without following a predefined sequence. Fully self-controlled systems can handle defects at the function or large project level. They promote autonomy and exploration at the cost of more complex coordination and reduced predictability. Another recent approach introduces conversational correction [19], in which the generation of fixes is intertwined with immediate feedback (testing, static analysis, compliance rules). This allows the correction to be carried out as an iterative, dialogue-driven process that facilitates the gradual refinement of fixes. And to limit the hallucinations generated by LLMs, some approaches use debate techniques [11], such as MultiAgents Debate (MAD) and Conquer-and-Merge Debate (CMD). For example, in the case of REpeated Clustering of Scores Improving the Precision (RECSIP) [13], this involves querying several models in parallel, then scoring and grouping their responses by similarity to achieve a more accurate and reliable output.

The same prompting techniques are applicable to all of the approaches outlined above. The most common prompt improvements are as follows:

- Few-shot prompting [8] includes similar examples of corrections in the prompt to enable the LLM to imitate the correction pattern;
- Retrieval Augmented Generation (RAG) [9] adds code snippets and domain facts from external resources to the prompt to ground the correction in concrete information;
- Analysis Augmented Generation (AAG) [7] injects reports extracted from static or dynamic analyses to guide the model toward the source of the problem.

2.3 Limitations of existing methodologies

Despite their promising performance, automatic correction approaches based on LLMs have several structural limitations. The limited size of the context window makes it difficult to consider large software projects, as the capability to include all necessary files and dependencies for a consistent overall correction is currently limited. Assessing the validity of the generated fixes is also a major challenge: while human expertise allows for a detailed evaluation of the functional correction, it remains time-consuming. Furthermore, the robustness of workflows is affected by the stochastic nature of LLMs: variability accumulates over the course of the steps, making it difficult to reproduce results and ensure consistency across a project. Finally, security and confidence issues remain. In particular, the generated fixes sometimes modify more code than necessary, which raises the central problem of quickly and reliably verifying the changes.

Existing automatic correction solutions remain largely incomplete. Some comprehensive development frameworks integrate compliance modules in the form of test components in the loop [15]. They aim to detect and correct violations early in the development process. However, the authors often offer only a limited level of detail on such modules and these approaches are used in simpler use cases where early application of standards from the development phase makes it possible to limit the scope of necessary corrections. Conversely, more academic

Table 1. Mapping of identified gaps in the literature to the proposed workflow components (Figure 1).

Aspect	Gap / Limitation	Proposed component
Rule coverage of corrections	No LLM-based approach covers a broad set of MISRA-C rules beyond simple patterns (Clava-MISRA).	LLM Correction: LLM framework with MISRA-C curated knowledge for the compliance of industrial-scale and mature software projects.
Large software projects	Limited context window size	Extract code snippet: Code objects affected by a violation are specifically identified to form the prompt.
Grounding & hallucination control	Hallucinations are reduced by LLM techniques but domain-specific grounding is unexploited for MISRA-C.	Corrections database + Developers & MISRA-C directives: Augmented generation with validated fixes, MISRA standard knowledge and curated developers directives.
Evaluation	LLM volatility makes it difficult to ensure consistency in both the results and the nature of the corrections.	Compliance regression tests + Extract code snippet: LLM modifications are limited to the selected code objects given in the prompt and compliance regression tests prevent the introduction of new violations. At this stage, human verification is still required solely to ensure code quality.

approaches to MISRA-C, such as Clava-MISRA, offer a correction loop applied to entire codebases. But for this project, corrections are limited to a restricted set of automatable rules, implemented statically (e.g. unused code, corrections to definitions and identifiers, library management...). Table 1 summarises how the proposed framework mitigates the previously identified limitations.

3 Proposed Method

We propose an LLM framework with a procedural approach for predictability and control. Figure 1 presents the overall architecture of the automated correction workflow. The system comprises three subsystems: (i) a *Knowledge specific database* module that assembles the contextual prompt (Section 3.2), (ii) a *Correction loop* block that iterates between an LLM-based corrector and a static analyser (Section 3.1), and (iii) the *Test-in-the-Loop* gate that validates each candidate fix before integrating them in the source code (Section 3.3). Design choices are motivated by the gaps identified in Table 1.

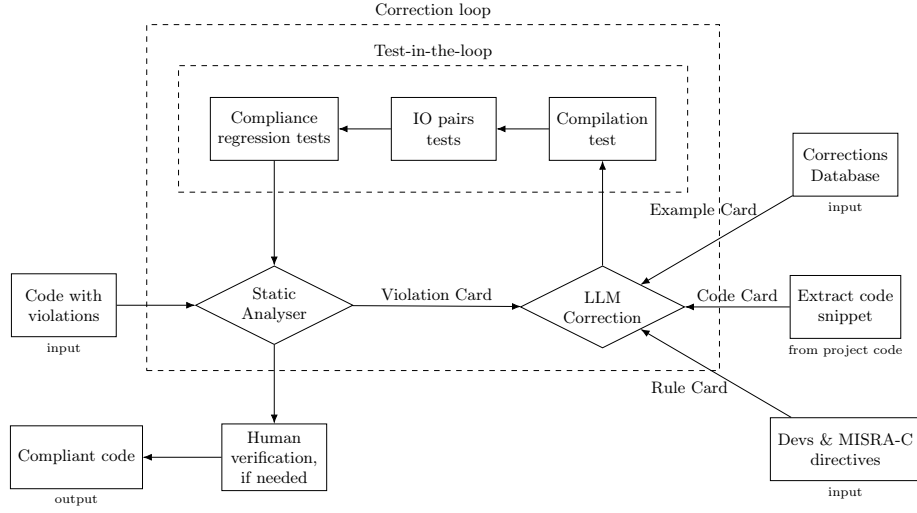


Fig. 1. Overview of the Automated Correction Workflow

3.1 Correction Loop

The correction loop alternates between a stochastic generative step and a deterministic verification step. At pipeline entry, the static analyser produces a compliance report on the current source code. The static analysis tool covers the majority of MISRA-C rules and provides reproducible verdicts. Corrections are performed at the function level, with each correction attempt targeting violations in a single function independently. To support batch execution, the framework groups independent functions that do not share referenced code objects, preventing overlap between correction tasks. This isolation enables independent batch processing and facilitates clearer identification of the root cause of potential errors detected during the validation stage (Section 3.3).

At each iteration, for each of the selected independent functions, the LLM is provided with an augmented prompt, the structure of which is illustrated in Figure 2 and explained in Section 3.2. The LLM is queried iteratively until it produces a response that is both syntactically parseable and semantically valid, i.e., suitable for automated use such as a complete corrected function together with its modified dependencies. The corrections are then processed by the test-in-the-loop module described in Section 3.3. Since the selected functions are independent, it is possible to determine precisely which corrections introduce or resolve a violation. The valid corrections are inserted in the source code while invalid ones are discarded. Following each iteration, the updated compliance report is used to identify the next set of functions and violations to correct.

The loop terminates when either no target violations remain or the pre-defined maximum number of iterations is reached.

Correction task + Rule Card

Make an appropriate correction on this code to avoid the MISRA-C violations described in lines 10, 27 and 30 of {my_modf}.
Return the whole corrected function and its modified dependencies.
Rule MISRA-C 10.4: Both operands of an operator shall have the same essential type category.
If possible, types of global variables or structure attributes shall be modified to avoid explicit casting in the function.

Normative
knowledge

Code Card + Violation Card

```
static double my_modf(double x0, double *iptr)
{
    /* more code */
    // Event misra_c_2012_rule_10_4_violation: Essential type
    // difference between "l" (signed) and "x - 1."(float).
    if (l <= (x + 1.0) && l >= (x - 1.0)) {
        break;
    }
    /* more code */
}
```

Violation
context

Example Cards

Example 1: `length * 0x4F` → `length * 0x4Fu` (*suffix correction*)
Example 2: `1 < 12` → `1u < 12` (*macro type change*)
Example 3: `(bss_end - bss_start)` → `(size_t)(...)` (*explicit cast*)

Empirical
knowledge

Fig. 2. Simplified structure of the augmented prompt used by the LLM to correct a function, illustrated with MISRA-C Rule 10.4. The four cards are described in Sec. 3.2.

3.2 Contextual Prompt

The MISRA-C specific knowledge module constructs a grounded prompt from four information sources, each materialised as a structured *card*—a self-contained, machine-readable information unit that the LLM consumes at correction time. Cards decouple knowledge acquisition from prompt assembly: each source is maintained independently as the project is being modified. The prompt builder selects and concatenates all available cards for each correction attempt. Cards are non-overlapping and should be used whenever available. Figure 2 illustrates the resulting prompt layout for a concrete MISRA-C Rule 10.4 violation; the Code Card and Example Cards, described below, are simplified for clarity in the prompt example.

Some of the information cards heavily rely on MISRA-C correction guidelines derived from previous industrial compliance activities. These guidelines capture knowledge produced by developers and compliance experts while correcting violations in earlier projects, including recurring remediation patterns and ac-

cepted correction strategies. They adapt standard requirements to project- and domain-specific correction practices and are defined separately for each standard and violation type. This accumulated knowledge is reused to contextualize the prompt and guide the LLM toward corrections consistent with prior compliance practices.

Within the pipeline, the static analysis stage produces the **Violation Card** by associating the static analysis report information with the corresponding parsed code objects within the project. When building the prompt, a precise contextual comment is inserted directly above the affected source line for each violation to help the model focus on the relevant code region.

To implement few-shot prompting, a curated set of validated correction pairs (i.e., source code with violations and their corresponding corrected versions) serves as the few-shot demonstration pool. Each entry is stored as an **Example Card** containing the simplified non-compliant code with its violation context (Violation Card of the example), the corrected code snippet, and the strategy used for the correction. Three to four simplified example cards have been extracted from the developers guidelines for each violation type or rule to provide domain- and standard-specific grounding. For each rule, the example cards are designed to illustrate a variety of resolution patterns and to highlight the preferred correction strategies. Since the examples are simplified, all rule-specific examples can be injected in the prompt without exceeding the context window.

Beyond the source lines flagged by the static analyzer, surrounding code is extracted and packaged into a **Code Card**: the enclosing function, relevant type definitions, and callers/callees up to a configurable depth. This is essential for rules that depend on data flow or type consistency (e.g., MISRA-C Rules 10.x on the essential type model). The compliance report highlights the affected source line, from which function dependencies—including structs, enums, unions, and global variables—can be readily traced, leveraging Abstract Syntax Tree (AST) parsing, and included in the prompt. Larger context cards provide richer code context but compete for limited context-window space. Therefore, the code objects closest to the violation are selected to form a minimal relevant context that balances understanding of the surrounding function with the code elements most likely to be modified. The selection method was validated for the MISRA-C standard on an industrial-scale corrected codebase, where it successfully identified the code objects modified in each function based on the precise source-line information reported by the static analysis tool. This technique enables the framework to scale to large codebases rather than being limited to small file-level examples. By leveraging static analysis, the card mitigates context-window limitations and directs the model’s attention to the relevant region of the code.

For a given standard, the full text of the applicable rule and its rationale, together with domain- and project-specific instructions extracted from correction guidelines, are assembled into a **Rule Card** for each standard rule. Thus, the model receives both examples with reasoning and also guidance on the preferred correction strategies associated with each violation. The context provided by the cards, together with the use of an LLM, enables correction of a diverse range of

rules beyond the simpler, automatable subset handled by tools such as Clava-MISRA.

The prompt builder concatenates all cards in a fixed order: the instruction header, the Rule Card, the Violation Cards embedded within the Code Card, and the Example Cards. The prompt targets a single violation type or rule at a time, and the model attempts to correct all violations of that rule detected in the function currently being corrected.

3.3 Test-in-the-Loop

Before final integration into the source code, the corrected version must pass a three-stage validation gate ordered by increasing cost.

First stage. For each correction attempt, which targets a specific function, a reduced compilation subproject is generated containing the modified code objects together with the dependent objects that reference or use them. Source files are reconstructed to include only the relevant objects required for compilation, while preserving the original project production-level compilation flags, include paths, and build configuration. This reduced project is then compiled instead of the full codebase to validate the generated correction. Any compilation failure immediately rejects the proposed fix, otherwise the modifications are integrated into the project source code. This approach significantly reduces validation time on large industrial codebases, where full project compilation can be costly. It also enables parallel validation of independent correction attempts, since each function-specific subproject can be compiled and tested separately without invalidating unrelated corrections.

Second stage. A suite of functional unit tests, integration tests, or input-output pairs is then executed through a dedicated test executable. This step verifies that the correction preserves the program’s behavior, and not merely compliance-preserving. The quality of the test suite becomes the main limiting factor: insufficient coverage may fail to detect regressions, while overly strict tests may reject semantically equivalent rewrites. Depending on the size of the source code, this tests can be too time-consuming to be integrated in the iterative correction loop, and not all projects provide a pre-existing test suite when performing compliance corrections. In the case of ExeBench, the dataset provides real-world executable C functions together with input/output examples and a test executable, making the integration of these unit tests into the framework relatively straightforward.

Third stage. Using the same strategy as in the first stage, a dedicated subproject is generated for each function correction to analyze the modified code without requiring static analysis of the entire codebase. This verification ensures that no new standard violations are introduced into code objects potentially affected by the modification and that the targeted violations are resolved. This step enforces the minimal-change principle, ensuring the fix resolves the targeted violations without expanding the non-compliance surface. The verification is performed by comparing the results with the compliance snapshot originally used to identify the violations and generate the prompts. The correction is ultimately

accepted only if it passes all three validation steps (when available) and reduces the number of violations without introducing any new ones.

4 Results

4.1 Dataset

To obtain initial results with the framework, we applied it to resolve MISRA-C violations in C functions from the ExeBench dataset. ExeBench is particularly well suited for this evaluation because for subset of functions it provides a dedicated test executable along with real input-output pairs for each function. These tests enable automatic verification that proposed corrections preserve the original program semantics, ensuring the tests suite validation gate.

Since the ExeBench functions are restricted to single-file projects, the framework was additionally evaluated on a non-compliant version of an industrial Secure-IC codebase. The project contains approximately 840,000 lines of code across 2,733 C source and header files, characterizing a large-scale industrial software system. Despite the scale of the project, the average code complexity remains moderate, with a mean cyclomatic complexity of 4 and approximately 14 instructions per function. All framework components operated successfully at this scale, including specifically AST-based dependency extraction and subproject generation, compilation, and static analysis comparison between correction iterations.

4.2 Evaluation

The evaluation focuses on a subset of MISRA-C rules that are both frequently encountered in the ExeBench dataset and representative of issues commonly found in industrial code bases. In addition, we selected rules that can not be easily addressed using simple hard-coded code transformations, except for the rule 15.6 added as a comparison example. Many of these violations require contextual reasoning about types, expressions, or surrounding code structures, making them suitable candidates for an LLM-based correction approach.

The correction results for both ExeBench dataset and the industrial project are presented in Table 2. The targeted MISRA-C rules are the following:

- **10.1:** Operands shall not be of an inappropriate essential type
- **10.3:** The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category
- **10.4:** Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category
- **14.4:** The controlling expression of an if statement and the controlling expression of an iteration-statement shall have essentially Boolean type
- **15.5:** A function should have a single point of exit at the end
- **15.6:** The body of an iteration-statement or a selection-statement shall be a compound-statement

Table 2. Framework performance per rule

Rules	ExeBench results					Industrial project results				
	LLM Attempts		MISRA-C Violations			LLM Attempts		MISRA-C Violations		
	Total	Validated	Total	Corrected	Rate	Total	Validated	Total	Corrected	Rate
10.1	904	124	1162	557	48%	168	20	117	64	55%
10.3	901	232	704	485	69%	218	27	60	47	78%
10.4	616	235	1131	745	66%	238	111	358	255	71%
14.4	300	163	230	212	92%	20	10	16	13	81%
15.5	465	152	342	274	80%	78	20	91	35	38%
15.6	300	290	531	512	96%	40	31	55	45	82%
18.4	108	82	245	212	87%	44	14	29	27	93%
8.10	160	87	91	87	96%	/	/	/	/	/
Total	3754	1365	4436	3084	70%	806	233	726	486	67%

Rules : MISRA-C rule types

LLM Attempts Total : Number of correction attempts

LLM Attempts Validated : Number of corrections accepted by the framework

MISRA-C Violations Total : Number of targeted violations

MISRA-C Violations Corrected : Number of resolved violations

MISRA-C Violations Rate : Correction success rate

- **18.4**: The +, -, += and -= operators should not be applied to an expression of pointer type
- **8.10**: An inline function shall be declared with the static storage class

On ExeBench, rule 15.6 achieves an almost perfect resolution rate as expected. The current framework version is able to solve 3084 (70%) of the 4436 targeted violations with 3754 correction attempts. These results suggest that the framework requires further refinement for rule 10.1 violations, while otherwise offering a solid baseline performance.

The results on the Secure-IC project exhibit some variability due to the limited number of correction attempts and targeted violations considered at this stage. The lower violation resolution rate observed for rule 15.5 may indicate that the selected functions contain more complex return-flow and dataflow structures than the ExeBench C functions used during evaluation. Overall, the framework was able to solve 486 (67%) of the 726 targeted violations with 806 correction attempts on this project.

From an industrial perspective, the key evaluation criterion is the final violation resolution rate rather than the acceptance rate of individual correction attempts, since iterative generation may be necessary before a valid correction is obtained. What ultimately matters is the framework’s ability to resolve targeted violations reliably within the automated correction loop. But, we further analyzed the rejected correction attempts on the ExeBench dataset, where input/output test pairs are available, to characterize the main sources of failure. On the 3754 correction attempts, 432 (11%) of the candidates fail during compilation, typically due to incorrect or incomplete transformations generated by the model. Among the corrections that successfully compile, 809 (22%) fail the

IO pairs validation tests, indicating that the modification alters the program behavior regardless of the compliance result. Finally, 1148 (31%) correction candidates fail due to compliance issues either because the original violation remain unresolved or because the modification introduce a new violation. The remaining 1365 (36%) accepted corrections resolve 3084 MISRA-C violations. Overall, these results underscore the challenge of generating repairs that are syntactically correct, compliant with the relevant rules, and semantically faithful to the original code.

To illustrate these observations, we provide two representative examples of MISRA-C rule 15.5 violations. The first example, shown in Listing 1.1, is taken from the ExeBench dataset, where the framework successfully produces a correct resolution in a relatively simple control-flow context. In contrast, the second example, shown in Listing 1.2, originates from the industrial Secure-IC codebase and highlights a significantly more complex scenario, where multiple status checks, conditional early returns, and chained function calls increase the difficulty of correctly restructuring the control flow. This rule appears to be more sensitive than others to function-level complexity, particularly in the presence of intricate return-flow and dataflow patterns. In particular, the function exhibits repeated status propagation and nested dependency handling, which further complicates the correction process compared to the more linear patterns observed in ExeBench.

Listing 1.1. ExeBench C function with MISRA-C 15.5 violations

```

1 beat (int firstfigures[], int secondfigures[], int index) {
2     int i;
3     for (i = 0; i < 2; ++i) {
4         if (firstfigures[index] == secondfigures[i]) {
5             secondfigures[i] = 1;
6
7             // Event misra_c_2012_rule_15_5_violation
8             return;
9         }
10    }
11    for (i = 0; i < 2; ++i) {
12        if (i != index) {
13            if (firstfigures[index] == firstfigures[i]) {
14                firstfigures[i] = 1;
15
16                // Event misra_c_2012_rule_15_5_violation
17                return;
18            }
19        }
20    }
21 }
```

Listing 1.2. Redacted industrial C function with MISRA-C 15.5 violations

```
1 struct struct1 fct(const struct struct2 *curve, const struct3 *x, const int
   y_lsb) {
2     struct struct1 pkreq;
3
4     pkreq = fct1(curve, VAR1);
5     if (pkreq.status != OKAY)
6         return pkreq;
7
8     struct struct4 var1;
9     pkreq.status = fct2(pkreq, curve, y_lsb, (struct struct5*)&var1);
10    if (pkreq.status != OKAY)
11        return pkreq;
12
13    int var2 = fct3(pkreq);
14    fct4(x, var1, var2);
15    fct5(pkreq.req);
16
17    return pkreq;
18 }
```

4.3 LLM setup and Execution Cost Analysis

The framework uses the Qwen3-Coder-30B-A3B-Instruct [18] model served through vLLM using probabilistic token sampling based on temperature and nucleus (top-p) sampling. A temperature of 0.4 is used to maintain controlled output diversity across successive correction attempts, enabling the exploration of alternative correction strategies when previous generations fail validation. Nucleus sampling is configured with a top-p value of 0.95 to preserve generation diversity while limiting unlikely outputs. These parameters were selected based on early empirical testing and preliminary experimentation, although more extensive evaluation and tuning will be required in future work.

The framework minimizes validation overhead by avoiding full-project compilation and static analysis after each correction attempts batch. A naive approach would directly integrate generated modifications into the original source tree and repeatedly rebuild and analyze the entire project, resulting in substantial computational costs on large industrial codebases. Instead, the framework generates reduced subprojects containing only the minimal set of code objects required to compile the corrected function together with its surrounding modifications. Similarly, static analysis is performed on a dedicated reduced project for each corrected function containing the code objects impacted by the correction and the objects depending on them. This targeted validation strategy significantly reduces compilation and analysis time while ensuring that the correction remains locally compliant and does not introduce new violations in nearby dependent code.

The LLM inference time and framework overhead remain comparable between the two approaches. However, in the naive validation strategy, rebuilding and analyzing the entire industrial project introduces a substantial additional cost, with compilation and static analysis requiring approximately 226 seconds per correction iteration. In contrast, for the same correction batch size, the proposed subproject-based validation approach reduces this cost to approximately

10 seconds for compilation, and 21 seconds for localized static analysis over the impacted code region. On the evaluated industrial project, the validation time is reduced from 226 to 31 seconds per iteration ($7.3\times$ faster). As project size increases, the expected speedup also increases, whereas the relative improvement decreases as the batch size grows.

5 Conclusion

In this work, we proposed a methodology for an unsupervised AI-based method for automatic code repair in the context of C language programs and MISRA-C compliance. Our results indicate that the proposed LLM-based approach is capable of automatically resolving the majority of MISRA-C violations commonly encountered in industrial C projects. While performance is strong across most rules, certain cases, such as rule 10.1, remain challenging and would benefit from further refinement of the correction strategies.

Future work will focus on improving the test-in-the-loop module. Since input-output pairs or semantic tests are not always available and can be time-consuming, we aim to investigate lighter approaches for semantic correctness verification based on statistical analysis and metrics on the modified code. We also have access to an industrial database of validated corrections, which will be used to compare human-written and model-generated fixes in order to assess how closely the generated corrections match industrial development practices. Future work will also involve the development of a dedicated evaluation and experimentation environment to systematically assess and refine correction strategies across different project configurations and standards. Leveraging this environment, we will explore different prompting strategies, prompt-construction techniques, and contextualization methods to improve correction quality and better align generated fixes with developer expectations, project-specific correction guidelines, and coding practices.

References

1. ISO 26262-6:2018 Road vehicles — Functional safety — Part 6: Product development at the software level (2018)
2. ANSSI: ANSSI Guidelines, Rules for secure C language software development (2022)
3. Armengol-Estapé, Jordi and Woodruff, Jackson and Brauckmann, Alexander and Magalhães, José Wesley de Souza and O’Boyle, Michael F. P.: ExeBench: an ML-scale dataset of executable C functions. In: *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. p. 50–59. MAPS 2022, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3520312.3534867>, <https://doi.org/10.1145/3520312.3534867>

4. Boyang Yang and Zijian Cai and Fengling Liu and Bach Le and Lingming Zhang and Tegawendé F. Bissyandé and Yang Liu and Haoye Tian: A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications (2025), <https://arxiv.org/abs/2506.23749>
5. Chih-Yuan Chen and Yung-An Fang and Guan-Ren Wang and Peng-Sheng Chen: A GCC-based checker for compliance with MISRA-C's single-translation-unit rules. *Connection Science* **35**(1), 2222934 (2023). <https://doi.org/10.1080/09540091.2023.2222934>, <https://doi.org/10.1080/09540091.2023.2222934>
6. Kirchner, Florent and Kosmatov, Nikolai and Prevosto, Virgile and Signoles, Julien and Yakobowski, Boris: Frama-C: A software analysis perspective. *Formal aspects of computing* **27**(3), 573–609 (2015)
7. Li, F., Jiang, J., Sun, J., Zhang, H.: Hybrid automated program repair by combining large language models and program analysis. *ACM Trans. Softw. Eng. Methodol.* **34**(7) (Aug 2025). <https://doi.org/10.1145/3715004>, <https://doi.org/10.1145/3715004>
8. Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G.: Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Comput. Surv.* **55**(9) (Jan 2023). <https://doi.org/10.1145/3560815>, <https://doi.org/10.1145/3560815>
9. Mansur, E., Chen, J., Raza, M.A., Wardat, M.: Ragfix: Enhancing llm code repair using rag and stack overflow posts. In: 2024 IEEE International Conference on Big Data (BigData). pp. 7491–7496 (2024). <https://doi.org/10.1109/BigData62323.2024.10825785>
10. Milena Luísa Pereira Gouveia: Source-to-Source Approach for MISRA-C. Master's thesis, Universidade do Porto (2025)
11. Petrovic, N., Zolfaghari, V., Schamschurko, A., Kirchner, S., Pan, F., Wu, C., Purschke, N., Velsh, A., Lebiada, K., Song, Y., Zhang, Y., Mazur, L., Knoll, A.: Survey of GenAI for Automotive Software Development: From Requirements to Executable Code (2025), <https://arxiv.org/abs/2507.15025>
12. Roberto Bagnara and Abramo Bagnara and Patricia M. Hill: The MISRA C Coding Standard and its Role in the Development and Analysis of Safety- and Security-Critical Embedded Software (2018)
13. Schamschurko, André and Petrovic, Nenad and Knoll, Alois Christian: RECSIP: REpeated Clustering of Scores Improving the Precision. In: Arai, K. (ed.) *Intelligent Systems and Applications*. pp. 115–127. Springer Nature Switzerland, Cham (2025)
14. Seacord, Robert: CERT C Secure Coding Standard (Oct 2008), <https://insights.sei.cmu.edu/library/cert-c-secure-coding-standard/>
15. Sven Kirchner and Alois C. Knoll: Generating Automotive Code: Large Language Models for Software Development and Verification in Safety-Critical Systems (2025), <https://arxiv.org/abs/2506.04038>
16. Synopsys: Coverity Prevent. <https://www.blackduck.com/static-analysis-tools-sast/coverity.html> (2025), accessed: 2026-06-22
17. Tao, Wei and Zhou, Yucheng and Wang, Yanlin and Zhang, Wenqiang and Zhang, Hongyu and Cheng, Yu: MAGIS: LLM-based multi-agent framework for GitHub issue ReSolution. In: *Proceedings of the 38th International Conference on Neural Information Processing Systems*. NIPS '24, Curran Associates Inc., Red Hook, NY, USA (2024)
18. Team, Q.: Qwen3 technical report (2025), <https://arxiv.org/abs/2505.09388>

19. Xia, Chunqiu Steven and Zhang, Lingming: Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. p. 819–831. ISSSTA 2024, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3650212.3680323>, <https://doi.org/10.1145/3650212.3680323>